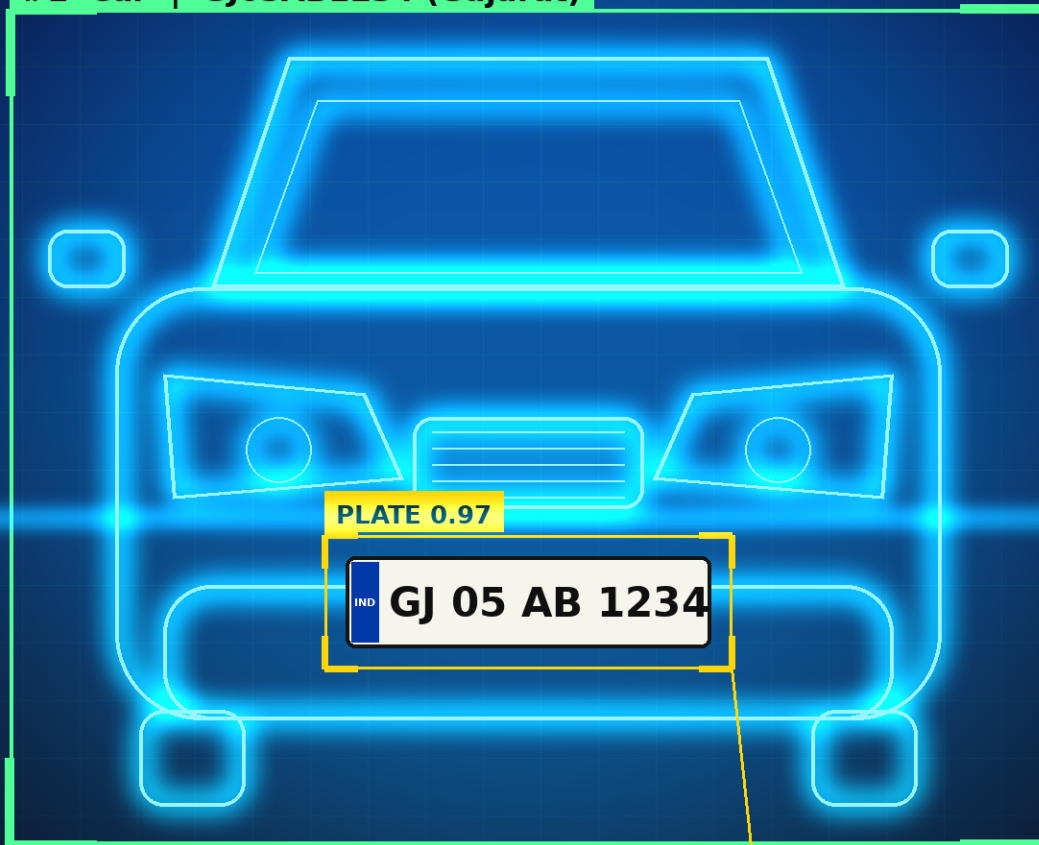


VEHICLE NUMBER PLATE DETECTION

Automatic Number Plate Recognition for Indian Vehicles

Photos • Videos • Multi-Object Tracking • OCR Voting

#1 Car | GJ05AB1234 (Gujarat)



GJ05AB1234

OCR: EasyOCR • conf 0.92

SHAH RISHABH ALPESHBHAI

23SS02IT181 | B.Sc. Information Technology
P P Savani University, School of Engineering

PROJECT REPORT

[github.com/shahrishabh1513-jsk/
/vehicle-number-plate-detection](https://github.com/shahrishabh1513-jsk/vehicle-number-plate-detection)

A Project Report

Submitted in partial fulfilment of the requirements
for the award of the degree of

Bachelor of Science in Information Technology

on

VEHICLE NUMBER PLATE DETECTION

Automatic Number Plate Recognition (ANPR) using YOLOv8 and EasyOCR

by

SHAH RISHABH ALPESHBHAI

23SS02IT181

Under the supervision of

Mr. SAURAV SINGH

Assistant Professor, School of Engineering



OCTOBER 2026

P P SAVANI SCHOOL OF ENGINEERING

P P SAVANI UNIVERSITY

NH NO.: 8, VILLAGE: DHAMDOD, TA. MANGROL, NEAR KOSAMBA, SURAT – 394 125. (GUJARAT).

GitHub: <https://github.com/shahrishabh1513-jsk/vehicle-number-plate-detection>

CERTIFICATE

This is to certify that the Project Report titled “**Vehicle Number Plate Detection**” submitted by **SHAH RISHABH ALPESHBHAI (23SS02IT181)** to the **P P SAVANI UNIVERSITY** for the partial fulfilment of the subject credit requirements is a bonafide work carried out by the student.

This is to further certify that I have been supervising the Project of **SHAH RISHABH ALPESHBHAI (23SS02IT181)**.

The contents of this report, in full or in parts, have not been submitted to any other Institute or University for award of any degree, diploma or titles.

Date:
Place: Surat

Mr. SAURAV SINGH
Assistant Professor, SoE

ACKNOWLEDGEMENT

This report would not have been possible without the guidance of my teachers, who were always there when I needed them the most. I take this opportunity to acknowledge them and extend my sincere gratitude for helping me make this report possible.

I wish to thank my faculty mentor **Mr. Saurav Singh**, Assistant Professor, School of Engineering. It has been an honour to learn under his mentorship. As my mentor, he constantly motivated me to remain focused on my goal. His observations and guidance helped me establish the overall direction of the project and move forward with learning in depth. His support at each juncture culminated in the successful completion of my project, **“VEHICLE NUMBER PLATE DETECTION – AUTOMATIC NUMBER PLATE RECOGNITION USING YOLOv8 AND EASYOCR.”**

I am also thankful to the faculty members of the department for their constant support and guidance.

I am thankful to the Dean, School of Engineering, for the initiative of including project work during the tenure of our study, allowing us to learn industry-standard technologies and helping us expand our professional horizons.

I also acknowledge the open-source communities behind **Ultralytics YOLOv8, EasyOCR, OpenCV, PyTorch, NumPy, pandas** and **Matplotlib**, as well as Google Colab for providing free GPU-enabled notebooks. Without these tools, a project of this nature could not have been completed by a single student.

Name of Student: SHAH RISHABH ALPESHBHAI (23SS02IT181)

ABSTRACT

The “**Vehicle Number Plate Detection**” project is an end-to-end **Automatic Number Plate Recognition (ANPR)** system developed in Python to detect vehicles, locate their licence plates, and read the registration number from photographs and videos. The primary objective is to turn raw road footage into structured, searchable data – a list of unique plates, the type of vehicle that carries each plate, the issuing state, and the confidence with which each plate was read – with no manual transcription.

The system is built as a modular pipeline. A pre-trained **YOLOv8** model (*yolov8s.pt*) detects four vehicle classes – car, motorcycle, bus and truck – while a second YOLOv8 model detects the licence plate itself. Each plate is cropped with several padding levels and passed through three image-enhancement variants (up-scaling, CLAHE contrast enhancement and Otsu binarisation) before being read by **EasyOCR** restricted to the characters A–Z and 0–9. For videos, **ByteTrack** assigns persistent vehicle IDs and a custom **IoU-and-distance plate tracker** follows each plate across frames, so that the final plate number is chosen by **confidence-weighted voting** over many OCR reads instead of trusting a single noisy frame.

A key contribution of the project is its **India-specific post-processing layer**. Using regular expressions for the standard state-series format (e.g. GJ05AB1234) and the Bharat (BH) series format (e.g. 22BH1234AA), a correction routine repairs common OCR look-alike mistakes such as O/0, I/1, S/5, B/8 and G/6, trims stray leading or trailing characters within a bounded edit budget, validates the state code against a table of Indian state and union-territory codes, and reports the full state name. The pipeline produces annotated images and MP4 videos with vehicle boxes, plate boxes, zoomed plate thumbnails and text labels, together with a *detected_plates.csv* report and built-in diagnostics that explain why a plate could not be read.

From a technical and professional perspective, the project demonstrates practical computer-vision engineering: model integration with automatic weight download and integrity checks, image pre-processing for OCR, domain-specific text correction, multi-object tracking, voting-based decision making, and reproducible reporting on Google Colab with GPU acceleration. The complete source code is available at the project’s GitHub repository.

CONTENTS

ACKNOWLEDGEMENT	4
ABSTRACT	5
CHAPTER 1: OVERVIEW OF PROJECT	9
1.1 Overview of the Project	9
1.2 Problem Statement	9
1.3 Objectives	9
1.4 Project Details	9
1.5 Applications	10
1.6 Development Methodology	10
CHAPTER 2: INTRODUCTION	11
2.1 Project Overview	11
2.2 Scope of the Project	11
2.3 Tools & Technology Used	11
2.4 Project Details: Module-wise Analysis	12
CHAPTER 3: LEARNING OUTCOMES & DISCUSSION	15
3.1 Learning Outcomes	15
3.2 Challenges Faced and Solutions	15
3.3 Glimpses of the Project	15
3.4 Limitations & Future Improvement Suggestions	16
CHAPTER 4: SYSTEM DESIGN & DIAGRAMS	17
4.1 Architecture Design	17
4.2 Data Flow Diagram (DFD Level 0 & Level 1)	17
4.3 Use Case Diagram	18
4.4 Class Diagram	19
4.5 Data Model (ER-style Diagram)	20
4.6 Algorithm Flowchart – Indian Plate Correction	20
CHAPTER 5: CODING	22
5.1 Project Directory & Folder Structure	22
5.2 Notebook Cell Map	22
5.3 Backend Coding – Detection and Models	22

5.4 Frontend / Image-Processing Coding	23
5.5 Output & Data Integration	23
CHAPTER 6: SOFTWARE TESTING	25
6.1 Unit Testing	25
6.2 Integration Testing	26
6.3 Compatibility and Performance Testing	26
6.4 Robustness, Security and Privacy Considerations	26
CHAPTER 7: SNAPSHOTS (Visual Walkthrough)	28
7.1 Step 1 – Install and Configure	28
7.2 Step 2 – Upload Photos and Videos	28
7.3 Step 3 – Models Load	28
7.4 Step 4 – Pre-processing of a Plate Crop	28
7.5 Step 5 – Quick Test and Annotated Output	28
7.6 Step 6 – Voting Chooses the Final Plate	29
7.7 Step 7 – Final Report Table	29
7.8 Step 8 – Watch and Download	30
CHAPTER 8: CONCLUSION AND FUTURE SCOPE	31
8.1 Conclusion	31
8.2 Future Scope	31
CHAPTER 9: GLOSSARY	32
REFERENCES & PROJECT REPOSITORY	33

LIST OF FIGURES

Fig 4.1 Architecture Design (Layered ANPR Pipeline)	17
Fig 4.2 Data Flow Diagram – Level 0 (Context Level)	18
Fig 4.3 Data Flow Diagram – Level 1 (Process Level)	18
Fig 4.4 Use Case Diagram	19
Fig 4.5 Class / Component Diagram	19
Fig 4.6 Logical Data Model of the Pipeline State and Output	20
Fig 4.7 Flowchart of the fix_indian() Algorithm	21
Fig 7.1 Pre-processing variants generated for OCR (synthetic example plate)	28
Fig 7.2 Schematic of the annotated output (vehicle box, plate box, label and zoomed plate)	29
Fig 7.3 Score voting over simulated OCR reads of one plate track	29

LIST OF TABLES

Table 1.1: Project Details	10
Table 1.2: Development Phases of the Project	10
Table 2.1: Technology Stack	12
Table 2.2: Configuration Parameters	14
Table 3.1: Challenges and Solutions	15
Table 4.1: Entity Attribute Mapping (Output Schema)	20
Table 5.1: Notebook Step Map	22
Table 6.1: Unit Test Results – fix_indian() and plate_state_name()	25
Table 6.2: Unit Test Results – score_text()	25
Table 6.3: Integration and System Test Cases	26
Table 7.1: Sample Format of detected_plates.csv (illustrative values)	29

CHAPTER 1:

OVERVIEW OF PROJECT

1.1 Overview of the Project

Every day, millions of vehicles pass through toll plazas, parking lots, residential societies, campuses and city roads. The only reliable public identifier of a motor vehicle in India is its **registration number plate**. Recording these numbers manually is slow, error-prone and impossible at scale. **Automatic Number Plate Recognition (ANPR)** – also called Automatic License Plate Recognition (ALPR) – uses computer vision and optical character recognition to do this automatically.

This project, “**Vehicle Number Plate Detection**,” is a personal, self-driven implementation of a complete ANPR system. It accepts one or more photographs and/or videos, detects every car, bike, bus and truck, finds the number plate on each vehicle, reads the text, corrects it to the Indian registration format, and exports the findings as annotated media and a CSV report. The project is delivered as a single, well-documented Google Colab / Jupyter notebook so that anyone can reproduce the results by clicking “Run all.”

1.2 Problem Statement

Reading number plates from real-world footage is difficult for several reasons: plates are small relative to the frame, vehicles move, lighting changes, images are blurred or compressed, plates are dirty or at an angle, and OCR engines confuse visually similar characters such as **O/0, I/1, S/5** and **B/8**. A naive “detect and read once” approach therefore produces unreliable text. The problem addressed here is:

“How can a single pipeline reliably detect vehicles and number plates in photos and videos, read the plate text despite OCR noise, and report clean, India-format registration numbers together with vehicle type, state and confidence?”

1.3 Objectives

- Detect vehicles (car, motorcycle, bus, truck) and number plates in both still images and videos using YOLOv8.
- Improve OCR accuracy by pre-processing every plate crop in multiple ways (up-scaling, CLAHE, Otsu thresholding) and by trying several crop paddings.
- Correct OCR errors with a rule-based post-processor for Indian state-series and Bharat (BH) series plates and map the state code to the full state / union-territory name.
- Track vehicles and plates across video frames and decide the final plate number by confidence-weighted voting.
- Produce annotated images / videos and a structured *detected_plates.csv* report with built-in diagnostics.
- Keep the whole system reproducible, configurable through a small set of constants, and runnable on a free Colab GPU.

1.4 Project Details

Field	Details
Project Title	Vehicle Number Plate Detection (Automatic Number Plate Recognition)
Student	SHAH RISHABH ALPESHBHAI
Enrollment No.	23SS02IT181
Programme	Bachelor of Science in Information Technology
University	P P Savani University, School of Engineering, Surat
Mentor	Mr. Saurav Singh, Assistant Professor, School of Engineering
Project Type	Personal / academic project (self-driven)
Domain	Computer Vision, Deep Learning, OCR, Intelligent Transportation
Language / Platform	Python 3, Jupyter / Google Colab (GPU recommended)
Core Libraries	Ultralytics YOLOv8, EasyOCR, OpenCV, PyTorch, NumPy, pandas, Matplotlib
Source Code	https://github.com/shahrishabh1513-jsk/vehicle-number-plate-detection
Main File	Number_plate_detection_FINAL.ipynb

Table 1.1: Project Details

1.5 Applications

- **Traffic enforcement:** automated identification of vehicles for e-challan systems and speed / red-light violations.
- **Toll and parking automation:** ticket-less entry and exit, vehicle logs and billing.
- **Gated communities, campuses and offices:** visitor and resident vehicle access control.
- **Security and investigation:** searching footage for a particular registration number.
- **Fleet and logistics management:** tracking vehicle arrivals at depots and warehouses.

1.6 Development Methodology

The project followed an iterative, notebook-first prototyping approach. Each stage was implemented as a separate notebook step, tested with a quick visual check, and then merged into the full pipeline. Table 1.2 summarises the development phases.

Phase	Activity	Outcome	Status
1	Study of ANPR, YOLO object detection and OCR techniques; analysis of the Indian registration-plate format	Requirement understanding	Completed
2	Environment setup on Google Colab; automatic dependency installation	Reproducible environment	Completed
3	Integration of vehicle and plate detectors with automatic weight download and integrity checks	Working detection stage	Completed
4	OCR pipeline: crop padding, 3 image variants, EasyOCR with character allow-list	Raw plate text	Completed
5	Indian-format correction, state lookup and scoring	Clean plate text + state	Completed
6	Video tracking (ByteTrack + custom PlateTracker) and confidence-weighted voting	Stable per-plate result	Completed
7	Annotation, CSV reporting, diagnostics and download	Deliverable outputs	Completed
8	Testing, documentation and GitHub publication	Final notebook & report	Completed

Table 1.2: Development Phases of the Project

CHAPTER 2:

INTRODUCTION

2.1 Project Overview

The project is a complete ANPR pipeline that follows the sequence **Input → Detect vehicles → Detect plates → Pre-process → OCR → Correct → Track & vote → Annotate & export**. It is intentionally built in two complementary modes that share the same core functions:

- **Photo mode** – every image is analysed once, but each plate crop is read three times with different paddings (4 %, 8 % and 15 %) so that the best result can be kept.
- **Video mode** – every frame is processed, vehicles are tracked with ByteTrack, plates are tracked with a custom tracker, OCR is run on every second frame (up to 30 reads per plate), and the plate text is decided by voting.

Both modes produce the same outputs: an annotated file with coloured vehicle boxes, yellow plate boxes, state-aware text labels and a zoomed thumbnail of each plate, plus one consolidated CSV report for all uploaded files.

2.2 Scope of the Project

- **Input:** multiple files in one run; images (jpg, jpeg, png, bmp, webp, tif/tiff) and videos (mp4, avi, mov, mkv, webm, m4v, wmv, flv, mpg, mpeg).
- **Detection:** four vehicle classes from the COCO dataset (car, motorcycle, bus, truck) and licence plates.
- **Recognition:** English alphanumeric OCR with Indian-format correction (state-series and BH-series) and state-name lookup.
- **Reporting:** annotated media, *detected_plates.csv*, per-run diagnostics and an OCR debug log.
- **Out of scope:** model training / fine-tuning, real-time camera streaming, database storage and a web front-end (see Future Scope in Chapter 8).

2.3 Tools & Technology Used

The technologies below were chosen so that the system is accurate, free to use, easy to install and runnable on a student-grade setup.

Technology	Version / Model	Role in the Project
Python 3	3.x (Colab default)	Core programming language of the entire pipeline
Ultralytics YOLOv8	yolov8s.pt (small)	Vehicle detection and ByteTrack-based vehicle tracking; loads the plate detector
Licence-plate detector	license_plate_detector.pt	YOLOv8 weights specialised in locating number plates
EasyOCR	English reader	Deep-learning OCR that reads text from the plate crops
OpenCV (cv2)	4.x	Image/video I/O, resizing, CLAHE, Otsu threshold, drawing
PyTorch	with CUDA if available	Deep-learning backend used by YOLOv8 and EasyOCR
NumPy / pandas	–	Array maths and the final tabular report / CSV export
Matplotlib / IPython	–	Visual previews, galleries and embedded video playback
Regular expressions (re)	standard library	Validation and correction of Indian plate formats
Google Colab	T4 GPU runtime	Free cloud GPU notebook environment, file upload / download
FFmpeg	system tool	Re-encodes the output video to H.264 so it plays in browsers
Git & GitHub	–	Version control and public hosting of the project

Table 2.1: Technology Stack

2.3.1 YOLOv8 (You Only Look Once, version 8)

YOLOv8 is a state-of-the-art, single-stage object detector that predicts bounding boxes and class probabilities in one forward pass, making it fast enough for video. The project uses the **small (s)** variant for vehicles because it offers a good balance between speed and accuracy, and a second YOLOv8 model, trained specifically for licence plates, to locate the plate inside the scene.

2.3.2 EasyOCR

EasyOCR is a ready-to-use deep-learning OCR library based on a text-detection stage (CRAFT) followed by a recognition network. In this project the reader is created once for English and called with an *allow-list* of the 36 characters A–Z and 0–9, which removes punctuation and lower-case noise at the source.

2.3.3 OpenCV and image enhancement

OpenCV supplies all image operations. Three variants of each plate crop are generated: (1) a cubic-interpolated up-scaled colour image, (2) a CLAHE (Contrast Limited Adaptive Histogram Equalisation) grey-scale image that reveals characters in uneven lighting, and (3) an Otsu-thresholded binary image that separates dark characters from the plate background.

2.3.4 ByteTrack and the custom PlateTracker

ByteTrack, integrated in Ultralytics, gives each vehicle a persistent ID across frames. Because the plate detector is a separate model, a small custom **PlateTracker** class matches plate boxes between frames using Intersection-over-Union (IoU) and centre-distance, allowing the OCR reads of the same plate to be grouped together.

2.4 Project Details: Module-wise Analysis

The notebook is organised into ten steps. The pipeline can be understood as eight functional modules, described below with their features, technical implementation and purpose.

2.4.1 Input Handling Module

- **Key Features:** Accepts any mix of photos and videos; Colab file-upload widget with a manual *INPUT_PATHS* fallback for local Jupyter; automatic type detection by extension or by trying to decode the file.
- **Technical Implementation:** *media_kind()* classifies each file as image / video, *first_frame()* reads a preview, and a thumbnail grid (up to six) is shown before processing. Unreadable files are skipped with a warning.
- **Purpose:** To make the system easy to use and robust against wrong or corrupt uploads.

2.4.2 Vehicle Detection Module

- **Key Features:** Detects car, motorcycle (shown as “Bike”), bus and truck with confidence ≥ 0.30 at image size 960.
- **Technical Implementation:** YOLOv8s restricted to COCO classes {2, 3, 5, 7}; for videos the *track()* call with *bytetrack.yaml* and *persist=True* supplies stable vehicle IDs.
- **Purpose:** To associate each plate with a vehicle and to report the vehicle type.

2.4.3 Plate Detection Module

- **Key Features:** Locates plates at a low confidence of 0.25 and a large inference size of 1280 so that small, distant plates are not missed.
- **Technical Implementation:** A second YOLOv8 model (*license_plate_detector.pt*) is downloaded automatically from a primary and a fallback URL; *weights_ok()* verifies size and ZIP integrity, otherwise the user is asked to upload their own weights.
- **Purpose:** To find the exact region that contains the registration number.

2.4.4 Pre-processing and OCR Module

- **Key Features:** Three image variants per crop, multi-padding in photo mode, two-line plate handling and a character allow-list.
- **Technical Implementation:** *ocr_variants()* scales small crops toward 320 px width (max 4×) and returns up-scaled, CLAHE and Otsu images; *read_pieces()* runs EasyOCR and, when text boxes are spread vertically (> 30 % of height), sorts them top-row-then-bottom-row so that two-line plates are joined correctly.
- **Purpose:** To maximise the chance that at least one variant produces a correct reading.

2.4.5 Indian Format Correction Module

- **Key Features:** Repairs look-alike characters, trims stray characters, validates the format and state code, and names the state.
- **Technical Implementation:** *fix_indian()* tries six standard layouts (state 2 letters, RTO 1–2 digits, series 1–3 letters, 4-digit number) plus the BH layout, within a maximum of three edits, preferring candidates with a valid state code and the fewest edits (Section 4.6).
- **Purpose:** To convert noisy OCR text into valid registration numbers.

2.4.6 Scoring, Tracking and Voting Module

- **Key Features:** Each candidate string receives a score = OCR confidence $\times$ format bonus (1.0 for a valid Indian format, 0.6 for 6–10 characters, 0.4 otherwise).
- **Technical Implementation:** Scores of identical strings are accumulated per tracked plate; *best_text()* returns the string with the highest accumulated score; a minimum score of 0.10 and a vote threshold (*MIN_VOTES*) filter weak reads.
- **Purpose:** To make the final plate number stable and trustworthy even when individual reads are wrong.

2.4.7 Annotation and Visualisation Module

- **Key Features:** Colour-coded vehicle boxes (car, bike, bus, truck), yellow plate boxes, labels like “#3 Car | GJ05AB1234 (Gujarat)”, and a sharpened zoomed plate thumbnail pasted next to each plate.
- **Technical Implementation:** *put_label()*, *paste_thumbnail()* and *enhance_for_display()* (cubic up-scaling plus unsharp mask) inside *process_frame()*; drawing sizes scale with image resolution.
- **Purpose:** To let a human verify the result at a glance.

2.4.8 Reporting and Export Module

- **Key Features:** Consolidated pandas table, *detected_plates.csv*, plate-crop gallery with raw OCR for debugging, diagnostic messages, MP4 re-encoding and one-click ZIP download.
- **Technical Implementation:** Rows are de-duplicated per (source, plate); FFmpeg converts *mp4v* output to H.264 / yuv420p; videos under 25 MB are embedded in the notebook; outputs are zipped when more than two files exist.
- **Purpose:** To deliver usable, shareable results and make failures explainable.

2.4.9 Configuration Parameters

All tunable behaviour is controlled by a small block of constants at the start of the notebook (Table 2.2).

Parameter	Value	Meaning
VEHICLE_CONF / VEHICLE_IMGSZ	0.30 / 960	Minimum confidence and inference size for vehicle detection
PLATE_CONF / PLATE_IMGSZ	0.25 / 1280	Minimum confidence and inference size for plate detection
MIN_PLATE_W / IMAGE_MIN_PLATE_W	35 px / 20 px	Smallest plate width to attempt OCR (video / image)
OCR_EVERY_N	2	In videos, run OCR on every 2nd frame to save time
MAX_READS	30	Maximum OCR reads per tracked plate in video
MIN_PLATE_SCORE / MIN_VOTES	0.10 / 1	Minimum score and votes to accept a read
INDIAN_FORMAT	True	Enable Indian-plate correction and scoring
IMAGE_CROP_PADS	(0.04, 0.08, 0.15)	Three crop paddings tried in photo mode
PLATE_THUMB_H / GALLERY_THUMB_H	70 / 140 px	Height of the on-image and gallery plate thumbnails
DEVICE	GPU (0) or “cpu”	Chosen automatically with <i>torch.cuda.is_available()</i>

Table 2.2: Configuration Parameters

CHAPTER 3:

LEARNING OUTCOMES & DISCUSSION

3.1 Learning Outcomes

Building this project from scratch required combining ideas from deep learning, image processing, text processing and software engineering. The following competencies were achieved:

- **Applied Computer Vision:** I learned how single-stage detectors such as YOLOv8 work in practice – confidence thresholds, inference size, class filtering and the trade-off between speed and accuracy.
- **OCR Engineering:** I learned that OCR quality depends as much on *pre-processing* as on the model. Up-scaling, CLAHE, Otsu thresholding, character allow-lists and multi-padding crops significantly change what the OCR sees.
- **Domain-Specific Post-processing:** Writing regular expressions and a bounded-edit correction algorithm for Indian plates taught me how domain knowledge can repair machine-learning output.
- **Object Tracking and Voting:** Using ByteTrack, IoU-based matching and score voting showed me how temporal information turns unreliable single-frame predictions into reliable decisions.
- **Robust Software Design:** Auto-installing libraries, validating downloaded weights, providing fallback URLs, skipping unreadable files and giving diagnostic messages made the notebook resilient and user-friendly.
- **Data Reporting:** I used pandas to de-duplicate, order and export results, and Matplotlib to build previews and galleries.
- **Cloud and Version Control:** I learned to run GPU workloads on Google Colab and to publish and document a project on GitHub.

3.2 Challenges Faced and Solutions

Challenge	Solution Implemented
OCR confuses similar characters (O/0, I/1, S/5, B/8, G/6)	<code>fix_indian()</code> position-aware correction using TO_LET / TO_DIG maps and the Indian layout
Plates are small and blurry in distant vehicles	Plate detector at 1280 px, up-scaling to ≈ 320 px width, CLAHE and Otsu variants, multi-padding crops
Single-frame OCR errors in video	Plate tracking + accumulation of scores over up to 30 reads, then voting
Two-line plates (bikes) read in wrong order	<code>read_pieces()</code> groups text boxes into top and bottom rows before joining
Plate-detector weights unavailable or corrupted	Two download URLs, <code>weights_ok()</code> integrity check and manual upload fallback
Output video does not play in browser	FFmpeg re-encode to H.264 with yuv420p pixel format
Slow processing on CPU	Automatic GPU detection; OCR only every N-th frame; per-plate read cap

Table 3.1: Challenges and Solutions

3.3 Glimpses of the Project

- **The Detection Phase:** Setting suitable thresholds so that distant plates are found without producing many false boxes (PLATE_CONF 0.25, inference size 1280).
- **The Recognition Phase:** Comparing raw OCR output with corrected output through the plate-crop gallery and the OCR debug log, which showed exactly which characters were being mis-read.

- **The Tracking Phase:** Fine-tuning the tracker gap (20 frames), the IoU / distance rule and the voting scores so that one physical plate does not split into several identities.

3.4 Limitations & Future Improvement Suggestions

Limitations:

- 1 Pre-trained plate detector:** the plate detector is a third-party pre-trained model; the project has not fine-tuned it on a dedicated Indian plate dataset.
- 2 No benchmark yet:** the project has not been evaluated on a labelled dataset, so precision / recall figures are not reported in this document.
- 3 Format rules:** the regular expression expects a two-digit RTO code, so single-digit codes such as “DL8CAF5030” are not recognised as valid (observed in Chapter 6). Over-correction is possible when characters are missing.
- 4 Offline batch processing:** the pipeline processes uploaded files; it is not a live RTSP / webcam system.
- 5 Hardware:** a GPU runtime is strongly recommended; CPU execution works but is slow.
- 6 Language:** only English (Latin) characters are read; regional-script and non-standard fancy plates are out of scope.

Future Improvement Suggestions:

- Fine-tune YOLOv8 on a large Indian plate dataset (HSRP, two-line, EV green plates) and measure precision, recall and character-level accuracy.
- Replace or complement EasyOCR with a plate-specific recogniser (e.g. CRNN, PaddleOCR or a Transformer-based model).
- Allow 1–2 digit RTO codes and add a state-code check inside scoring to avoid false corrections.
- Wrap the pipeline in a Streamlit / Flask web application with a database back-end (see Chapter 8).

CHAPTER 4:

SYSTEM DESIGN & DIAGRAMS

4.1 Architecture Design

The system follows a **layered pipeline architecture** in which each layer has one responsibility and passes its result to the next layer. This separation of concerns makes it easy to replace a single layer – for example, swapping EasyOCR for another recogniser – without changing the rest of the system.

- **Layer 1 – Input & Settings:** file upload, media-type detection and the configuration constants.
- **Layer 2 – Detection:** YOLOv8s finds vehicles (with ByteTrack IDs in videos) and the licence-plate YOLOv8 model finds plates.
- **Layer 3 – Recognition:** crop with padding, three image variants, EasyOCR, Indian-format correction and scoring.
- **Layer 4 – Tracking, Voting & Output:** PlateTracker groups reads, voting selects the best text, and annotated media and CSV are produced.

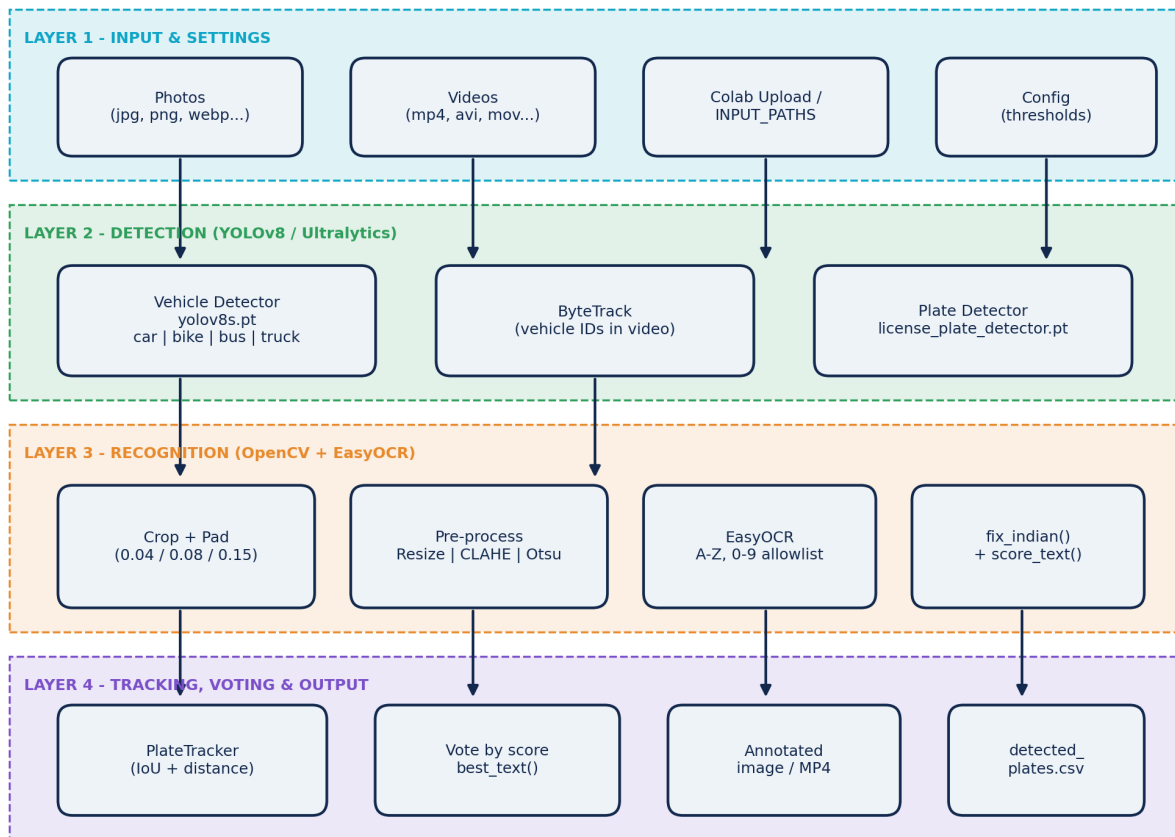


Fig 4.1 Architecture Design (Layered ANPR Pipeline)

4.2 Data Flow Diagram (DFD Level 0 & Level 1)

DFD Level 0 (Context Diagram) shows the system as a single process. The user supplies photos or videos; the system loads the pre-trained model weights and returns plate numbers, states, annotated media and a CSV report.

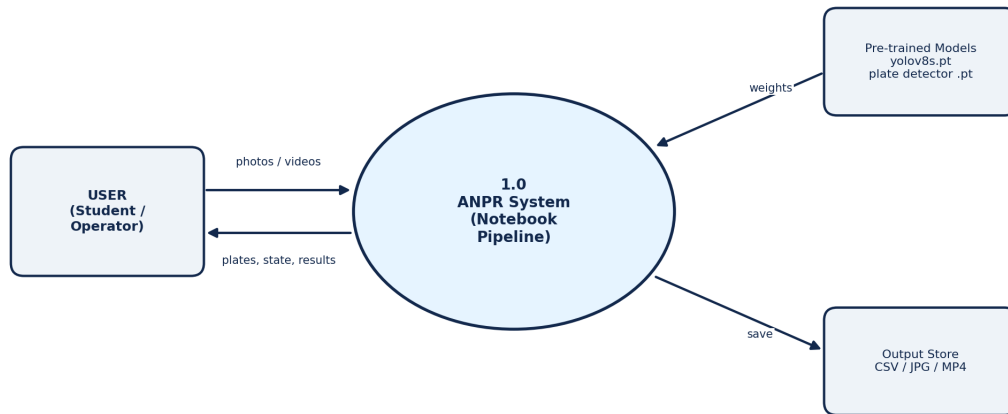


Fig 4.2 Data Flow Diagram – Level 0 (Context Level)

DFD Level 1 (Process Diagram) decomposes the system into six processes: (1.0) Input Validation, (2.0) Vehicle Detection, (3.0) Plate Detection, (4.0) OCR & Correction, (5.0) Tracking & Voting and (6.0) Reporting & Annotation. The data stores are the model weights (D1), the in-memory track state (D2) and the exported CSV (D3).

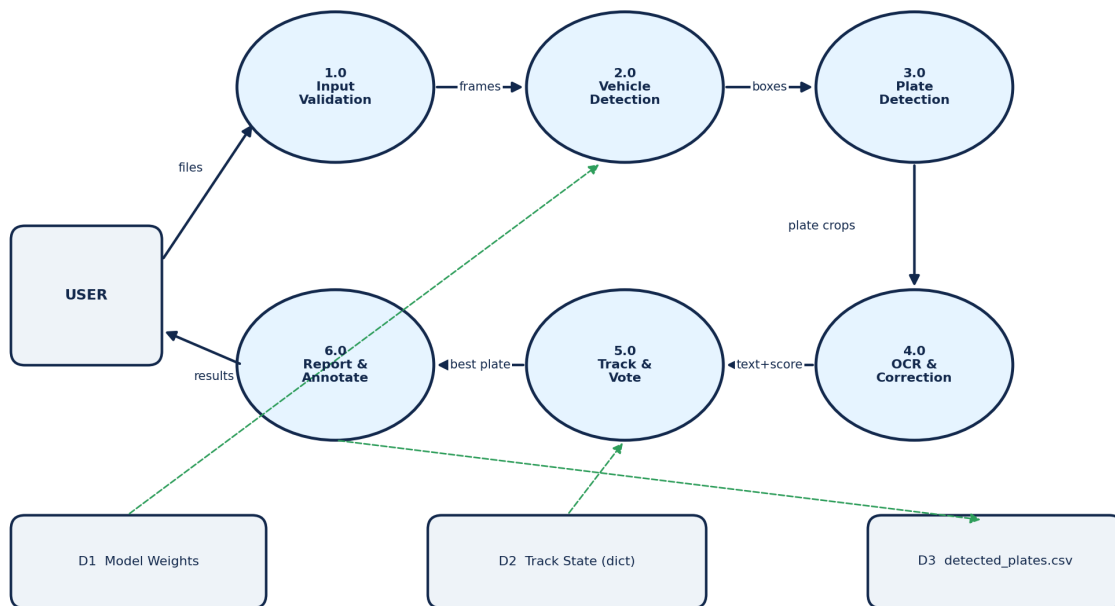


Fig 4.3 Data Flow Diagram – Level 1 (Process Level)

4.3 Use Case Diagram

The user is the primary actor. Uploading media, viewing annotated output and exporting the CSV are user-initiated use cases. Detection, OCR, correction, state identification and tracking are performed by the AI models acting as secondary actors inside the system boundary.

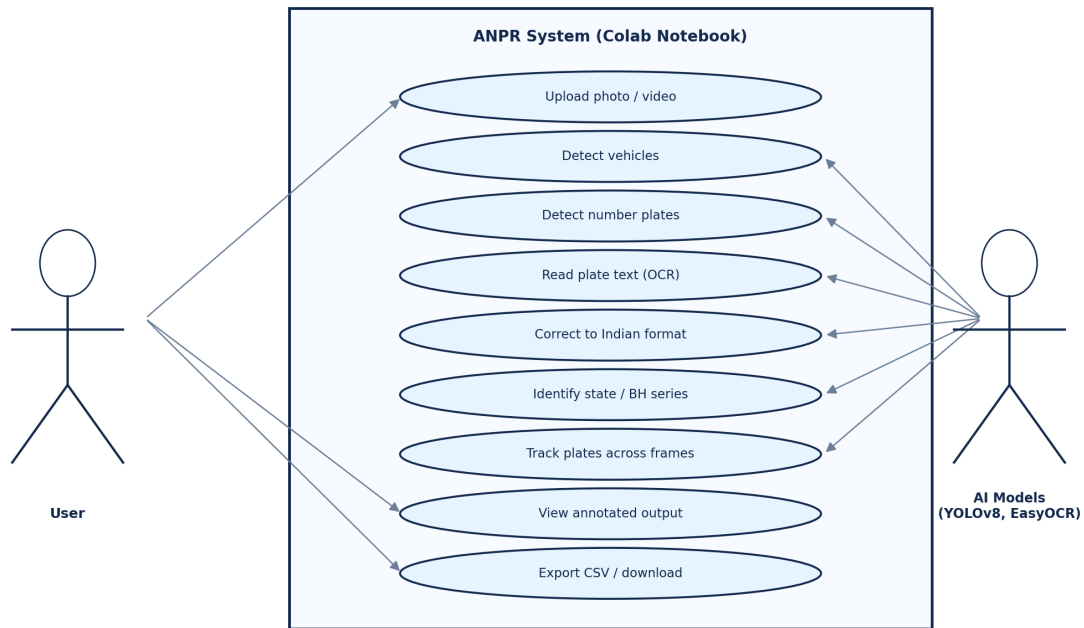


Fig 4.4 Use Case Diagram

4.4 Class Diagram

Although the notebook is mostly function-based, it contains one explicit class (**PlateTracker**) and a well-defined per-plate state record. The diagram below maps these structures together with the two function modules (OCR & Format Module and Processing Engine) and the configuration constants.

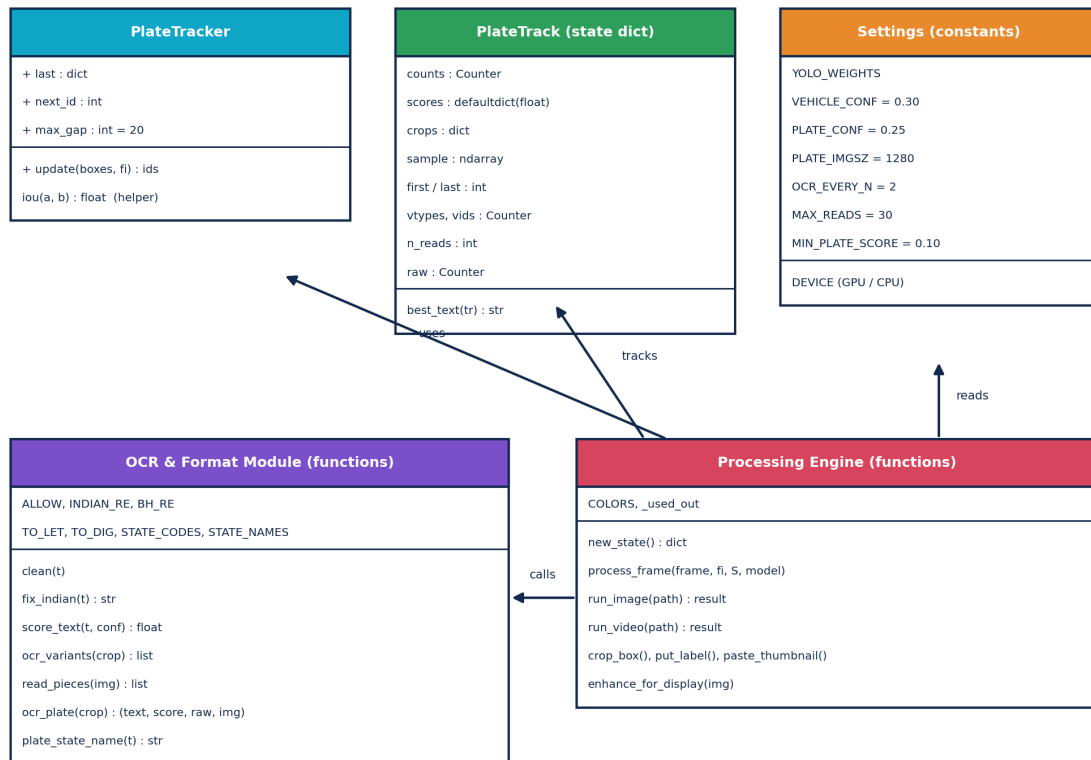
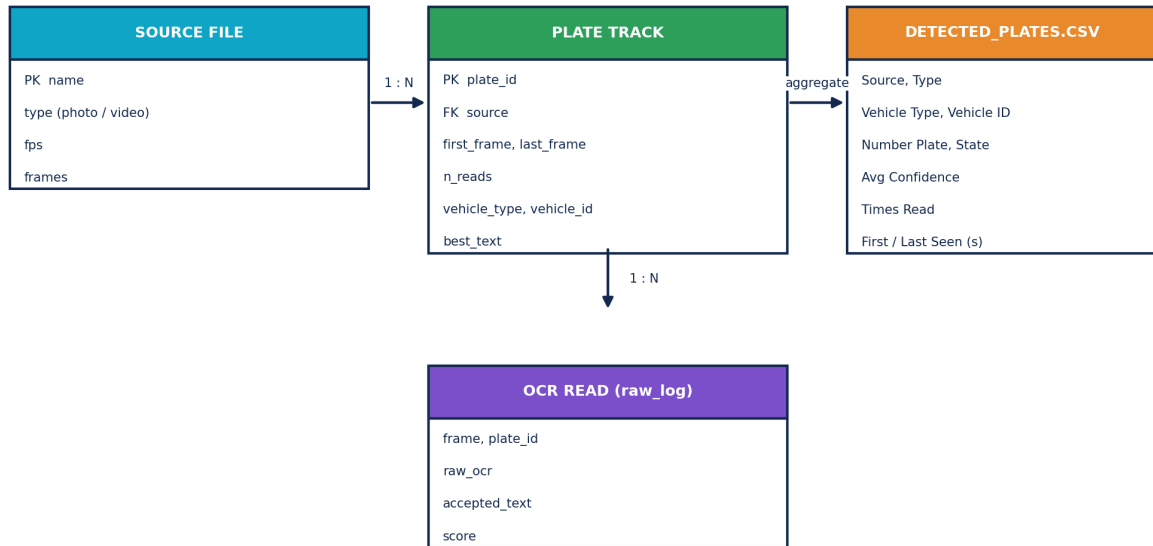


Fig 4.5 Class / Component Diagram

4.5 Data Model (ER-style Diagram)

The system has no external database, but it maintains structured data in memory and exports it to CSV. A source file has many plate tracks; each plate track has many OCR reads; and the tracks are aggregated into the rows of *detected_plates.csv*.



Logical data model of the pipeline's in-memory state and exported CSV

Fig 4.6 Logical Data Model of the Pipeline State and Output

4.5.1 Data Dictionary – detected_plates.csv

Column	Type	Description
Source	text	Name of the uploaded photo / video
Type	text	“photo” or “video”
Vehicle Type	text	Car, Bike, Bus, Truck or Vehicle (most frequent type that contains the plate)
Vehicle ID	integer	ByteTrack ID (photo mode: running index)
Number Plate	text	Final, corrected registration number
State	text	State / UT name, BH-series label or “-”
Avg Confidence	float	Accumulated score ÷ number of reads
Times Read	integer	How many OCR reads produced this text
First Seen (s) / Last Seen (s)	float	Time range in video (“-” for photos)

Table 4.1: Entity Attribute Mapping (Output Schema)

4.6 Algorithm Flowchart – Indian Plate Correction

The *fix_indian()* function is the heart of the domain-specific logic. If the text already matches a valid format with a known state code, it is returned unchanged. Otherwise the algorithm trims up to two characters from each end, fits the remainder into one of the standard layouts, replaces look-alike characters according to the expected letter / digit position, and keeps the candidate that has a valid state code and the fewest total edits (maximum three).

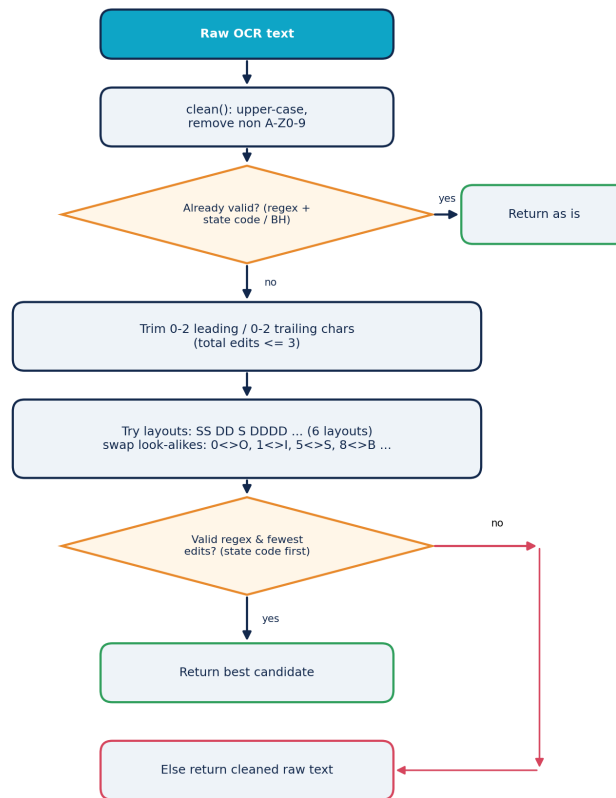


Fig 4.7 Flowchart of the `fix_indian()` Algorithm

CHAPTER 5:

CODING

Coding is the implementation phase where the architecture, data-flow and data-model designs are transformed into a working software product. The implementation is a single, well-structured notebook (*Number_plate_detection_FINAL.ipynb*) divided into ten numbered steps so that it can be run top-to-bottom on Google Colab.

5.1 Project Directory & Folder Structure

The GitHub repository contains the notebook. When it is run, the notebook creates or downloads the files shown below in the working directory.

```
vehicle-number-plate-detection/
|
|-- Number_plate_detection_FINAL.ipynb  <- the complete project (10 steps)
|
`-- (generated / downloaded at run time)
    |-- yolov8s.pt                      vehicle detector (auto-downloaded by Ultralytics)
    |-- license_plate_detector.pt       plate detector (auto-downloaded, validated)
    |-- annotated_<name>.jpg            annotated photo outputs
    |-- annotated_<name>.mp4            annotated video outputs (H.264)
    |-- detected_plates.csv             consolidated plate report
    `-- anpr_results.zip                all outputs zipped (when more than 2 files)
```

5.2 Notebook Cell Map

Step	Title	What the code does
1	Install libraries	Checks for ultralytics and easyocr with importlib; pip-installs only what is missing (safe to re-run)
2	Imports & settings	Imports libraries, defines thresholds, vehicle classes, display names, and selects GPU / CPU
3	Upload photos / videos	Colab upload widget, media_kind() / first_frame(), thumbnail preview, validation
4	Load models	Downloads and validates plate weights, loads YOLO vehicle + plate models, EasyOCR reader and CLAHE
5	Helper functions	Text cleaning, regex, state tables, fix_indian(), scoring, OCR variants, PlateTracker, drawing helpers
6	Quick test	Runs detection + OCR on the photos / 6 sample video frames before the full run
7	Processing engine	process_frame(), run_image(), run_video(), state container and output naming
8	Run on everything	Loops over uploaded files, prints diagnostics and previews
9	Final plates	Builds the pandas table, saves CSV, shows plate gallery and OCR debug log
10	Watch & download	FFmpeg re-encode, embedded playback, ZIP and Colab download

Table 5.1: Notebook Step Map

5.3 Backend Coding – Detection and Models

5.3.1 Configuration and model loading

```
YOLO_WEIGHTS = "yolov8s.pt"
VEHICLE_CLASSES = {2: "car", 3: "motorcycle", 5: "bus", 7: "truck"}
VEHICLE_CONF, VEHICLE_IMGSZ = 0.30, 960
PLATE_CONF, PLATE_IMGSZ = 0.25, 1280
DEVICE = 0 if torch.cuda.is_available() else "cpu"

def weights_ok(path):
    return (os.path.exists(path) and os.path.getsize(path) > 1_000_000)
```

```

        and zipfile.is_zipfile(path))

yolo_v = YOLO(YOLO_WEIGHTS)          # vehicles
yolo_p = YOLO(PLATE_WEIGHTS)         # plates
reader = easyocr.Reader(["en"], gpu=(DEVICE == 0))
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(4, 4))

```

5.4 Frontend / Image-Processing Coding

5.4.1 Plate pre-processing variants

```

def ocr_variants(crop):
    h, w = crop.shape[:2]
    scale = min(max(1.0, 320.0 / max(w, 1)), 4.0)
    big = cv2.resize(crop, None, fx=scale, fy=scale,
                     interpolation=cv2.INTER_CUBIC) if scale > 1 else crop
    gray = cv2.cvtColor(big, cv2.COLOR_BGR2GRAY)
    g_clahe = clahe.apply(gray)
    _, g_bin = cv2.threshold(cv2.GaussianBlur(gray, (3, 3), 0), 0, 255,
                             cv2.THRESH_BINARY + cv2.THRESH_OTSU)
    return [big, g_clahe, g_bin]

```

5.4.2 Indian-format validation and scoring

```

INDIAN_RE = re.compile(r"^[A-Z]{2}\d{2}[A-Z]{0,3}\d{4}$")
BH_RE = re.compile(r"^\d{2}BH\d{4}[A-Z]{2}$")
TO_LET = {"0":"0", "1":"1", "I":"2", "Z":"5", "S":"6", "8":"B", "4":"A"}
TO_DIG = {"0":"0", "Q":"0", "D":"0", "I":"1", "L":"1", "Z":"2", "S":"5",
          "G":"6", "B":"8", "A":"4", "T":"7"}

def score_text(t, conf):
    t = clean(t)
    if not 4 <= len(t) <= 12: return 0.0
    if sum(c.isdigit() for c in t) < 2: return 0.0
    if INDIAN_RE.match(t) or BH_RE.match(t): bonus = 1.0
    elif 6 <= len(t) <= 10: bonus = 0.6
    else: bonus = 0.4
    return conf * bonus

```

5.4.3 Plate tracker (IoU + distance matching)

```

class PlateTracker:
    def __init__(self, max_gap=20):
        self.last, self.next_id, self.max_gap = {}, 1, max_gap

    def update(self, boxes, fi):
        ids, used = [], set()
        for b in boxes:
            best, best_s = None, 0.0
            size = max(b[2]-b[0], b[3]-b[1], 1)
            for tid, (pb, pf) in self.last.items():
                if tid in used or fi - pf > self.max_gap: continue
                s = iou(b, pb)
                d = centre_distance(b, pb)
                if s < 0.05 and d > 2.0 * size: continue
                s2 = s + max(0.0, 1 - d / (2.0 * size)) * 0.5
                if s2 > best_s: best, best_s = tid, s2
            if best is None: best = self.next_id; self.next_id += 1
            used.add(best); self.last[best] = (b, fi); ids.append(best)
        return ids

```

(Code shortened for readability; the full version is in the repository. The centre-distance calculation is written inline in the original.)

5.5 Output & Data Integration

After processing, all plate tracks are aggregated into a pandas DataFrame, sorted, de-duplicated on (Source, Number Plate) and exported to CSV. The key lines are shown below.

```

rows.append({"Source": res["name"], "Type": "photo" if is_img else "video",
            "Vehicle Type": DISPLAY_NAME.get(tr["vtypes"].most_common(1)[0][0], "Vehicle"),
            "Number Plate": text, "State": plate_state_name(text) or "-",
            "Avg Confidence": round(tr["scores"][text] / tr["counts"][text], 2),
            "Times Read": tr["counts"][text], ...})

```

```
df = (df.sort_values("Times Read", ascending=False)
      .drop_duplicates(["Source", "Number Plate"])
      .sort_values(["_o", "First Seen (s)", kind="stable"])
      df.to_csv("detected_plates.csv", index=False)
```

Source code repository: <https://github.com/shahrishabh1513-jsk/vehicle-number-plate-detection>

CHAPTER 6:

SOFTWARE TESTING

Testing ensures that the pipeline behaves as expected, handles faulty input gracefully and produces trustworthy results. For this project the most valuable tests concern the text-correction and scoring logic (which is deterministic) and the behaviour of the full pipeline on different inputs.

6.1 Unit Testing

Unit testing verified the smallest components in isolation. The helper functions `fix_indian()`, `plate_state_name()` and `score_text()` were extracted directly from the project notebook and executed on controlled inputs. The results in Tables 6.1 and 6.2 are the actual outputs of that code (confidence for scoring was fixed at 0.90).

Raw OCR input	Situation tested	Corrected output	State	Score
GJ05AB1234	letter O instead of digit 0	GJ05AB1234	Gujarat	0.90
6J05AB1234	digit 6 instead of letter G	GJ05AB1234	Gujarat	0.90
GJ05A81234	digit 8 instead of letter B	GJ05AB1234	Gujarat	0.90
1GJ05AB1234	extra leading character	GJ05AB1234	Gujarat	0.90
GJ05AB1234X	extra trailing character	GJ05AB1234	Gujarat	0.90
MH12DE143B	letter B instead of digit 8	MH12DE1438	Maharashtra	0.90
22BH1234AA	Bharat (BH) series plate	22BH1234AA	BH-series (All-India)	0.90
DL8CAF5030	single-digit RTO code	DL8CAF5030	-	0.54
KA01AB123	one digit missing	KA01A8123	Karnataka	0.90
XX99ZZ0000	unknown state code	XX99ZZ0000	-	0.90

Table 6.1: Unit Test Results – `fix_indian()` and `plate_state_name()`

Input text	Situation tested	Score
AB	too short (< 4 chars)	0.00
ABCDEFGH	fewer than 2 digits	0.00
GJ05AB1234	valid Indian format	0.90
ABC12345	plausible, non-standard	0.54
1234567890123	too long (> 12 chars)	0.00

Table 6.2: Unit Test Results – `score_text()`

Observations from the unit tests

- **Look-alike repair works:** 0→O, 6→G and 8→B were corrected in the expected positions; extra leading / trailing characters were trimmed, and BH-series plates were accepted unchanged. The state table covers 38 state / union-territory codes (OD and OR both map to Odisha).
- **Single-digit RTO codes are not handled:** “DL8CAF5030” is a genuine Delhi format, but because the regular expression requires two RTO digits, it received only the 0.6 partial bonus (score 0.54) and no state name.

- **Over-correction risk:** “KA01AB123” (one digit missing) was forced into “KA01A8123”, which is a false but format-valid plate. This is a known weakness of any rule-based corrector and is listed as a future improvement.
- **Unknown state codes:** “XX99ZZ0000” matches the regular expression, so it earns the full format bonus, although no state name is returned. Adding a state-code check to `score_text()` would remove this loophole.

6.2 Integration Testing

Integration testing focuses on the communication between modules: detector → cropper → OCR → corrector → tracker → reporter. The following scenarios define the test procedure used with the notebook’s built-in diagnostics (Step 8) and the quick-test cell (Step 6).

ID	Scenario	Expected behaviour
IT-01	Single photo with one clearly visible car	One vehicle box, one plate box, plate text and state in label; one CSV row
IT-02	Photo with several vehicles	Each plate is associated with the smallest vehicle box that contains its centre
IT-03	Video with a moving vehicle	Same vehicle ID kept by ByteTrack; one plate track; final text chosen by voting; First / Last Seen filled
IT-04	Mixed upload (photos + videos)	All files processed in order; one combined CSV; ZIP created when more than 2 outputs
IT-05	Non-media or corrupt file	File skipped with a warning; remaining files still processed
IT-06	Plate detector finds no plate	Diagnostics message suggests lowering PLATE_CONF or using a closer / sharper shot
IT-07	Plate found but OCR unreadable	Diagnostics message explains the plate is too small / blurry; crops shown in gallery
IT-08	Plate weights download fails	Fallback URL is tried; then manual upload prompt is shown

Table 6.3: Integration and System Test Cases

6.3 Compatibility and Performance Testing

- **Runtime compatibility:** the notebook detects Google Colab (*google.colab*) and falls back to the `INPUT_PATHS` list in a local Jupyter environment; downloads switch to “Saved locally” outside Colab.
- **Hardware:** the same code runs on GPU (`DEVICE = 0`) or CPU. On CPU it prints a warning that processing will be slow.
- **Performance controls:** OCR every 2nd frame, a maximum of 30 reads per plate, and a minimum plate width of 35 px (video) reduce unnecessary computation.
- **File formats:** 7 image extensions and 10 video extensions are recognised, and unknown extensions are probed by trying to decode the file.

6.4 Robustness, Security and Privacy Considerations

- **Download integrity:** downloaded plate weights are accepted only when larger than 1 MB and a valid ZIP archive (PyTorch checkpoint), which prevents loading an HTML error page as a model.
- **Defensive coding:** OCR calls are wrapped in `try / except`; invalid crops are skipped; frame reading and writing are guarded; per-file failures are printed with a traceback without stopping the entire run.

- **Privacy:** number plates are personal data under many privacy regimes. All processing happens inside the user's own notebook session, no data is sent to a third-party service by the pipeline (only model weights are downloaded), and the project is intended for educational use with footage the user is entitled to process.
- **Ethical use:** ANPR outputs should be verified by a human before any enforcement action, because OCR errors can attribute a violation to the wrong vehicle.

CHAPTER 7:

SNAPSHOTS (Visual Walkthrough)

This chapter provides a step-by-step visual and functional guide to the notebook, from upload to final report. Figures in this chapter are produced with the project's own pre-processing and scoring code on a synthetic example plate ("GJ05AB1234") or are schematic illustrations of the annotated output.

7.1 Step 1 – Install and Configure

The first cell silently installs *ultralytics* and *easyocr* only if they are missing and prints "Libraries ready." The second cell imports the libraries, sets all constants and prints the device in use ("GPU" when CUDA is available, otherwise a CPU warning).

7.2 Step 2 – Upload Photos and Videos

In Colab a **Choose Files** button appears. After upload, each file is classified as a photo or video, its resolution (and for videos fps, frame count and duration) is printed, and a grid of up to six preview thumbnails is displayed. The summary line reads, for example, "3 file(s) ready: 2 photo(s), 1 video(s)."

7.3 Step 3 – Models Load

The plate detector is downloaded if not present, validated, and loaded together with the vehicle model and EasyOCR. The cell finishes by printing "Models ready" with the class names of the plate detector.

7.4 Step 4 – Pre-processing of a Plate Crop

Every plate crop is converted into three variants before OCR. Figure 7.1 shows the real output of the notebook's *ocr_variants()* logic on a deliberately small, noisy synthetic plate crop.

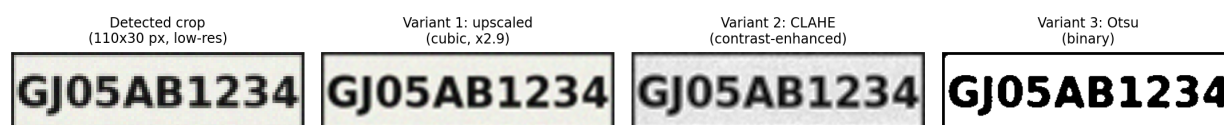


Fig 7.1 Pre-processing variants generated for OCR (synthetic example plate)

7.5 Step 5 – Quick Test and Annotated Output

Before the full run, a quick-test cell draws yellow plate boxes and shows the reading above each image. The full engine then draws vehicle boxes in class-specific colours (car, bike, bus, truck), a yellow plate box, the label with the state name, and a sharpened zoomed thumbnail of the plate, as illustrated schematically in Figure 7.2.

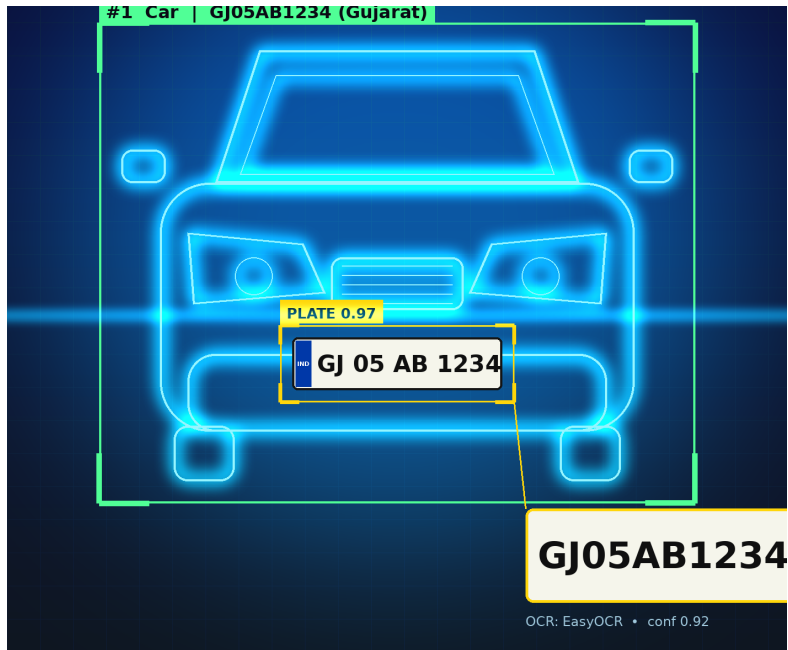


Fig 7.2 Schematic of the annotated output (vehicle box, plate box, label and zoomed plate)

7.6 Step 6 – Voting Chooses the Final Plate

For videos, each plate track accumulates many OCR reads. Figure 7.3 shows the effect of the scoring and voting logic using eight simulated raw reads of one plate (including O/O, G/6 and B/8 confusions and one truncated read), processed with the project's real `fix_indian()` and `score_text()` functions.

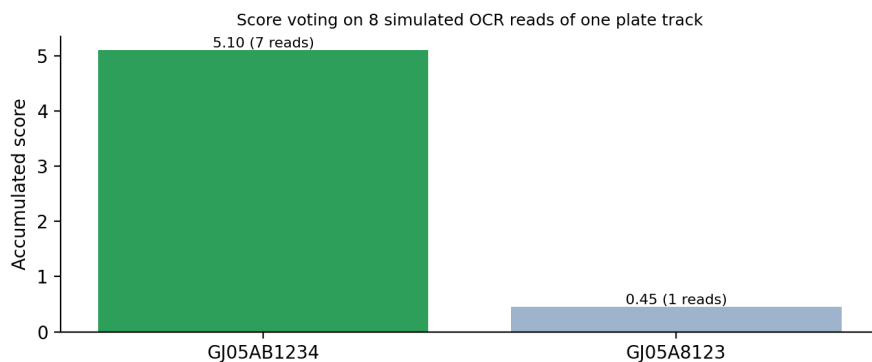


Fig 7.3 Score voting over simulated OCR reads of one plate track

7.7 Step 7 – Final Report Table

The report lists one row per unique plate per file and is saved as `detected_plates.csv`. Table 7.1 shows the **format** of the report with illustrative values (not real measurements).

Source	Type	Vehicle Type	Vehicle ID	Number Plate	State	Avg Conf.	Times Read	First / Last (s)
road1.mp4	video	Car	3	GJ05AB1234	Gujarat	0.73	7	1.2 / 4.8
road1.mp4	video	Bike	5	MH12DE1433	Maharashtra	0.61	5	2.0 / 5.1
photo1.jpg	photo	Truck	1	HR26DK8337	Haryana	0.68	3	- / -

Table 7.1: Sample Format of `detected_plates.csv` (illustrative values)

Below the table the notebook shows a gallery of the best plate crop for each plate with the corrected text and the raw OCR text, followed by the first 25 rows of the OCR debug log (frame, plate ID, raw OCR, accepted text

and score).

7.8 Step 8 – Watch and Download

Annotated videos are re-encoded to H.264 and, if smaller than 25 MB, played inside the notebook. All outputs and the CSV are then downloaded: individually when there are two files or fewer, or as *anpr_results.zip* otherwise.

CHAPTER 8:

CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

The “Vehicle Number Plate Detection” project successfully delivers a complete, reproducible ANPR pipeline for photographs and videos. By combining two YOLOv8 detectors, an EasyOCR recogniser, careful image pre-processing and an India-specific correction layer, the system converts noisy road imagery into clean registration numbers, vehicle types and states. The tracking-and-voting design shows how information from many frames can compensate for the weakness of any single OCR read.

Through this project I achieved several key milestones:

- **End-to-end integration:** I connected detection, tracking, OCR, text correction, annotation and reporting into one coherent workflow.
- **Problem solving:** I addressed real-world challenges such as small plates, look-alike characters, two-line plates, corrupted model downloads and browser-incompatible video output.
- **Professional practice:** I adopted configurable constants, defensive coding, diagnostics, unit testing of the core logic and public documentation on GitHub.

The unit tests also revealed honest limitations – single-digit RTO codes and over-correction – which provide a clear and realistic roadmap for improvement. In conclusion, the project meets its stated objectives and provides a strong foundation for a production-grade number-plate recognition system.

8.2 Future Scope

8.2.1 Custom-trained Detection and Recognition Models

Fine-tuning the plate detector on a large, labelled Indian dataset (including HSRP, two-line two-wheeler, commercial yellow and EV green plates) and training a plate-specific recognition network would improve accuracy in difficult conditions such as night, rain and motion blur. A proper benchmark with precision, recall and character-level accuracy should accompany this.

8.2.2 Smarter Validation

Extending the format rules to accept 1–2 digit RTO codes, validating RTO numbers per state, using edit-distance voting across candidates and learning common confusions from data would reduce false corrections.

8.2.3 Real-time and Edge Deployment

The pipeline can be adapted to live RTSP / webcam streams and exported to ONNX or TensorRT for NVIDIA Jetson-class edge devices, enabling on-site processing at gates, toll booths and traffic junctions.

8.2.4 Web Application and Database

A Streamlit or Flask / FastAPI front-end with a MySQL / PostgreSQL back-end could store vehicle logs, provide search by plate number, maintain watch-lists and visualise traffic statistics in a dashboard.

8.2.5 Additional Analytics and Privacy Features

Speed estimation, vehicle colour and make recognition, helmet detection for two-wheelers, automatic blurring of non-target plates and role-based access control would extend the system to smart-city and campus-security use cases while respecting privacy.

CHAPTER 9:

GLOSSARY

Term	Meaning
ANPR / ALPR	Automatic Number Plate Recognition / Automatic License Plate Recognition – reading vehicle registration numbers automatically from images or video.
YOLO	“You Only Look Once” – a family of fast single-stage object-detection neural networks; YOLOv8 is used here.
Ultralytics	Python package that provides YOLOv8 models, training, prediction and tracking tools.
COCO	Common Objects in Context – the dataset on which the pre-trained vehicle model was trained (car = 2, motorcycle = 3, bus = 5, truck = 7).
Bounding box	Rectangle (x1, y1, x2, y2) that marks the location of a detected object.
Confidence	A number between 0 and 1 expressing how sure a model is about a detection or a text reading.
Inference size (imgsz)	The image size to which a frame is resized before it is fed to the detector.
OCR	Optical Character Recognition – converting images of text into machine-readable characters.
EasyOCR	Open-source deep-learning OCR library supporting many languages; used with an English reader.
Allow-list	Restriction of OCR output to a given set of characters (here A–Z and 0–9).
OpenCV (cv2)	Open-source computer-vision library used for image reading, resizing, thresholding and drawing.
CLAHE	Contrast Limited Adaptive Histogram Equalisation – improves local contrast, helping OCR under uneven lighting.
Otsu thresholding	Automatic method to choose the threshold that converts a grey-scale image to black and white.
IoU	Intersection over Union – overlap ratio of two boxes; used to match plates between frames.
ByteTrack	Multi-object tracking algorithm that associates detections across frames to keep IDs stable.
Voting	Choosing the final value by summing scores over many readings and picking the highest.
RTO code	Regional Transport Office number (two digits) in an Indian registration, e.g. 05 in GJ05AB1234.
State-series format	Indian format: 2-letter state code, 2-digit RTO, 1–3 series letters, 4-digit number.
BH series	Bharat series registration (e.g. 22BH1234AA) valid across India without re-registration.
HSRP	High Security Registration Plate used on Indian vehicles.
Regular expression	A pattern language used to validate text formats, such as INDIAN_RE and BH_RE.
pandas / DataFrame	Python library and table structure used to build and export the final report.
CSV	Comma-Separated Values – a simple text file format for tables.
FFmpeg	Command-line tool used to re-encode videos to H.264 so that they play in browsers.
H.264 / yuv420p	Widely supported video codec and pixel format used for the final MP4.
Google Colab	Free cloud notebook service from Google with optional GPU runtime.
GPU / CUDA	Graphics processor and NVIDIA platform used to accelerate deep-learning computation.
DFD	Data Flow Diagram – shows how data moves between processes, stores and external entities.
PWA / ONNX / TensorRT	Technologies mentioned in future scope for web, portable-model and edge-GPU deployment.

REFERENCES & PROJECT REPOSITORY

- 1 Ultralytics, *YOLOv8 Documentation*. <https://docs.ultralytics.com>
- 2 JaiedAI, *EasyOCR* – Ready-to-use OCR. <https://github.com/JaiedAI/EasyOCR>
- 3 G. Bradski, *The OpenCV Library*, Dr. Dobb's Journal of Software Tools, 2000. <https://opencv.org>
- 4 J. Redmon, S. Divvala, R. Girshick, A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," IEEE CVPR, 2016.
- 5 Y. Zhang et al., "ByteTrack: Multi-Object Tracking by Associating Every Detection Box," ECCV, 2022.
- 6 K. Zuiderveld, "Contrast Limited Adaptive Histogram Equalization," *Graphics Gems IV*, Academic Press, 1994.
- 7 N. Otsu, "A Threshold Selection Method from Gray-Level Histograms," IEEE Trans. Systems, Man, and Cybernetics, 1979.
- 8 T.-Y. Lin et al., "Microsoft COCO: Common Objects in Context," ECCV, 2014.
- 9 Ministry of Road Transport and Highways, Government of India – vehicle registration mark and Bharat (BH) series notifications.
- 10 M. Z. Khan, *Automatic-License-Plate-Recognition-using-YOLOv8* (source of the pre-trained plate-detector weights referenced in the notebook). <https://github.com/Muhammad-Zeerak-Khan/Automatic-License-Plate-Recognition-using-YOLOv8>
- 11 PyTorch, NumPy, pandas, Matplotlib and Google Colab official documentation.

Project Source Code (GitHub)

<https://github.com/shahrishabh1513-jsk/vehicle-number-plate-detection>



Scan to open the repository